Budapest University of Technology and Economics
Faculty of Electrical Engineering and Informatics
Department of Artificial Intelligence and Systems Engineering

Training and Interpretation of a Neural Network Model for Traffic Object Detection

Bachelor's Thesis

Author
Péter Nyilas

Advisor
dr. Gábor Hullám

December 11, 2024

Contents

HALLGATÓI NYILATKOZAT

Alulírott *Nyilas Péter*, szigorló hallgató kijelentem, hogy ezt a szakdolgozatot meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző(k), cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövegét pedig az egyetem belső hálózatán keresztül (vagy autentikált felhasználók számára) közzétegye. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Budapest, 2024. december 1.

Nyilas Péter
hallgató

Kivonat

Ez a szakdolgozat egy mély neurális háló alapú rendszer fejlesztését mutatja be, amely képes felismerni és osztályozni különböző dinamikus közlekedési objektumokat menetrögzítő kamera által készített képeken. A dolgozat további célja a kialakított neurális hálózat eredményeinek interpretálása erre alkalmas magyarázó módszerek segítségével.

A feladat megvalósításához a Yolov8 modell alkalmazására került sor, amely egy konvolúciós mély neurális hálózat alapú objektumfelismerő algoritmus. Modell interpretálására modellfüggő és modellfüggetlen megoldásokat vizsgál a dolgozat, ismerteti ezek elméleti hátterét. Részletez három megoldást, amelyek a LIME, SHAP és az EigenCAM lettek.

A dolgozat tárgyalja a modell adatkezelésének, betanításának és interpretálásának folyamatát, valamint értekezik a gyakorlatot alátámasztó elméletről és elért eredményekről. Az elemzések során a dolgozat kitér a különböző magyarázómódszerek hiányosságaira és korlátaira, például a számítási igényekre, az értelmezések szubjektivitására vagy a módszerek specifikus alkalmazhatóságára a különféle modellek esetében.

Az eredmények alapján a dolgozat összegzi a tanulságokat, javaslatokat tesz az alkalmazási lehetőségek továbbfejlesztésére, és új kutatási irányokat vázol fel, amelyek további fejlődést hozhatnak a modellek magyarázhatóságának és megbízhatóságának területén.

Abstract

This thesis presents the development of a deep neural network based system capable of detecting and classifying different dynamic traffic objects in images captured by a motion capture camera. A further objective of the thesis is to interpret the results of the developed neural network using suitable explanatory methods.

To accomplish this task, the Yolov8 model is applied, which is a convolutional deep neural network based object recognition algorithm. Model-dependent and model-independent solutions for model interpretation are investigated and their theoretical background is described. It details three solutions, which are LIME, SHAP and EigenCAM.

The paper presents a discussion of the processes involved in the management, training and interpretation of models, with an examination of the theoretical principles and empirical outcomes that underpin these practices. In the course of the analysis, the paper addresses the shortcomings and limitations of the different methods of interpretation, such as computational demands, the subjectivity of interpretations or the specific applicability of the methods to different models.

Based on the results, the paper summarises the lessons learned, makes suggestions for further development of the applications, and outlines new research directions that could lead to further progress in the field of model explainability and reliability.

1 Introduction

The accurate detection of objects that are relevant to vehicle control is a critical task in the field of automotive and transportation systems. It is a particularly important aspect for the development of highly sophisticated autonomous driving systems, where numerous scenarios can arise with different types of objects to detect and react to. These objects can be either static (not moving), or dynamic (capable of moving). The object detection is implemented through the help of sensors using different physical phenomena, such as acoustic or electromagnetic waves. These sensors are mounted to the chassis of the autonomous vehicle.

One of the most widely used solution is the use of camera sensors as they are able to provide rich visual data that can be processed rapidly and can give the most information by itself. The main goal of this project was to develop a software capable of detecting dynamic traffic objects, while staying robust and reliable in various conditions. A fitting solution for this task was to utilise deep learning models, especially Convolutional Neural Networks, CNNs for short.

Nevertheless, the most significant limitation of these models is that they are frequently regarded as black boxes, lacking a straightforward correlation between input and output. As we might want to use these systems in safety critical applications, it is important to avoid unpredictable model-behaviour or mistrust from the stakeholders. High level of explainability should be the bases for trust and confidence between the system and its stakeholders. In accordance with this, the artificial intelligence-functional safety and AI systems ISO standard [1] devotes a chapter (8.3) to Degree of transparency and explainability. These needs led to the development of numerous model interpretation techniques, as evidenced by the work of a number of different researchers such as Yu Liang [2]. These techniques vary in their approach, but they are all aiming to facilitate an insight into the decision-making process of the model. In a unified manner, these methods are called eXplainable Artificial Intelligence methods, XAI for short.

The complexity of interpreting models in these domains arises from two key factors: the intricate nature of the data and the sheer size of models used. The aforementioned complexity makes it challenging to trace the decision-making process, which unfolds across numerous neurons and layers, presents a significant challenge. Nevertheless, it is essential for the identification of potential biases, the improvement of model performance, the fostering the trust of users and regulatory bodies. These factors are particularly important in the context of traffic object detection, where the consequences of model errors can be severe. This has heightened the importance of XAI solutions, with the aim of ensuring transparency for end users and developers alike.

As previously discussed in relation to other challenges facing the development of AI systems by Arun Das and Paul Rad in their article "Opportunities and Challenges in the Development of AI Systems" [3] the General Data Protection Regulation (GDPR) now requires that decisions made by automated systems be explainable to the user. Although the GDPR does not explicitly mention XAI and mainly focuses on data privacy, it seems probable that in the future, there will be a greater expectation of algorithmic transparency and clarification of AI systems.

It is therefore essential to enhance the interpretability of this project's model through the use of the aforementioned XAI methods such as SHAP, LIME and EigenCAM in order to maintain safety and ethical standards, given the nature of the application. Based on that the secondary objective of this project was to try and implement these interpretation methods and examine their effectiveness in the context of traffic object detection and try to instate a more transparent and reliable model.

1.1 Task refinement

Based on the different types of tasks I wanted to accomplish, I decided to separate them into two different tasks.

The principal objective of this project is to develop a software solution capable of detecting traffic objects in real time. This will entail an investigation into the fundamental principles of deep learning models for object detection, with a par-ti-cu-lar emphasis on the construction of a model based on the YOLOv8 architectural framework. The model will be trained on a dataset comprising images of urban en-vi-ron-ments, each of which has been annotated with traffic objects. Following training, the performance of the model will be evaluated based on its capacity to accurately detect traffic objects in real scenarios.

In addition to object detection, the secondary objective of the project is to enhance the interpretability of the model through the utilisation of Explainable AI (XAI) methodologies. The selection and implementation of both model-agnostic and model-specific interpretation techniques will be undertaken (they will be discussed and explained later), with a particular focus on understanding their underlying prin-cip-les and applying them to the traffic detection model. The effectiveness of these interpretation techniques will be evaluated based on their ability to provide insights into the rationale behind the model's predictions. This will ensure that the system is accurate and transparent in its decision-making processes.

The objective of this project is to develop a deep learning model for traffic object detection and to apply XAI methods to enhance the model's interpretability.

The two tasks were approached as discrete entities, and thus their explanations were presented separately. Initially, the object detection and the neural network's fulfilment of the task and its evaluation were discussed. Subsequently, the model's interpretation methods and their outcomes were evaluated.

The selection of dataset, model and interpretation methods was made with these goals in mind, with the intention of optimizing the model for the task in question, in line with the findings of the literature and previous studies.

1.2 Structure

The main structure of my thesis is outlined as follows.

The first major section focuses on the object detection component, which is divided into several subsections. It begins with a literature review on image processing techniques using deep convolutional neural networks. Following this, I provide a detailed description of the YOLO architecture, including an overview of the training process and the infrastructure used. Next, I cover data processing and representation with the Cityscapes dataset. I also give an overview of the training process. This section concludes with an evaluation of the model's performance.

The second main section addresses interpretation methods. It starts with an overview of model-agnostic and model-dependent interpretation techniques. Then, I present the implementation, application, and evaluation of the EigenCAM, LIME, and SHAP methods. Finally, this section analyses the results and draws conclusions regarding the effectiveness of the model interpretability solutions for detecting traffic objects.

2 Image Processing Deep Neural Networks

2.1 Overview of Convolutional Neural Networks

CNNs constitute a class of deep learning models that are commonly used for computer vision tasks including image classification, object detection, and semantic or instance segmentation. Given the ability of convolutional neural networks to learn spatial hierarchies of features and their various types of data representations (bounding box, segmentation mask, etc.), they are made ideal for the task of traffic object detection. Many recent advances in computer vision have been made possible by CNNs, including the development of models such as Yolo, Faster R-CNN, and SSD. A dozen of these models have been developed, each with its own unique architecture and capabilities. However, they all share the same fundamental principles gathered recently by Jiuxiang Gu in his 2018 work, "Recent advances in convolutional neural networks" [4].

The designation of the class is derived from the utilisation of convolutional layers, which apply filters to the input data in order to extract features. Additionally, convolutional neural networks comprise other layers, including pooling layers, fully connected layers, and so forth.

These models are employed in a multitude of applications, including the perception component in autonomous vehicles, surveillance systems, and medical imaging. In view of the fact that these applications are in alignment with the project's stated objectives, and given that the dataset in question is derived from a number of different urban traffic environments, it has been decided that these types of neural networks should be employed for the specific task of traffic object detection

The subsequent paragraphs will provide an overview of the essential components of convolutional neural networks.

Convolutional layers represent the fundamental building blocks of convolutional neural networks. Convolution operations are applied to the input data using filters (or in other word kernels) that process the input image. This process enables the model to identify local patterns such as edges, textures, and shapes. Each convolutional layer generates a set of feature maps, which indicate the presence of different types of these aforementioned features in the input. These feature maps are then passed to the subsequent layers for further processing, such operations as pooling or classification, to refine the information coded into the image. During the training phase, the parameters of the kernels are learned (their values are adjusted), through

backpropagation, allowing the network to enhance feature extraction for specific tasks.

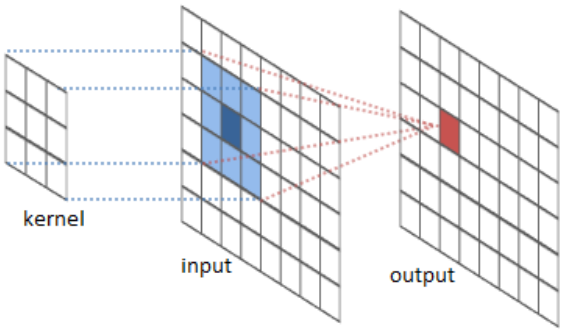


Figure 1: Convolutional layer operation [5]

Fully connected layers is used to describe a type of neural network in which each neuron is connected to every other neuron in the network's preceding layer, resulting in a dense connectivity pattern. This connectivity pattern is also referred to as a "dense layer" due to its dense connections on the semantic figure. They are typically located at the conclusion (HEAD) of convolutional neural network (CNN) architectures, where the final output is calculated. In these layers, as one neuron receives data from every neuron in the previous layer, each connection characterised by a specific trainable weight and each neuron possesses a unique trainable bias. This structure enables the model to integrate information from all features and make final predictions. Fully connected layers in image processing or object detection tasks are computing the output probabilities for each class based on the features extracted by the preceding layers. Regularization techniques, such as dropout and batch normalisation, are frequently employed in these layers to prevent overfitting.

Batch normalisation is a process that normalises the inputs of each layer in a neural network, ensuring that their mean is zero and their standard deviation is one. During training this normalisation is run on every mini-batch (small, randomly selected subset of training samples) of the data.

Dropout is a technique which involves the selecting of a random subset of neurons from the model's chosen layer, then ignoring them during training (dropping them out). This results in their exclusion from the whole learning process. The advantage of this process is that it helps the model to be resilient and less prone to overfitting, by forcing it to learn multiple independent representation of the data.

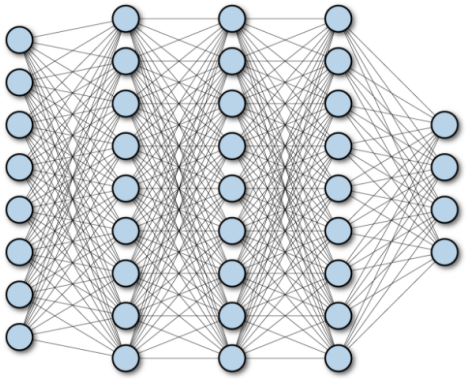


Figure 2: Fully Connected Layer semantics [5]

2.1.1 Sampling layers

Sampling layers are utilised to reduce the spatial dimensions of input data while maintaining its essential characteristics. This process encompasses techniques such as sub-sampling, strided convolutions, and global pooling. The following section provides an in-depth examination of several frequently employed sampling methods.

In the following paragraphs I will discuss some of the most important sampling layers.

Max Pooling is one of the most prevalent sub-sampling techniques employed in convolutional neural networks. In this approach, a sliding window processes the input feature map, and for each window, the maximum value is selected. This operation reduces the dimensionality of the feature map, preserving the most notable features (such as edges or textures) while discarding those of lesser relevance. Max pooling is particularly effective in object detection tasks where maintaining the strongest activations is of paramount importance, as evidenced by the findings of GU. [4].

Average Pooling , similarly to maximum pooling, average pooling utilises a sliding window; however, rather than calculating the maximum value, it computes the average. This results in a more gradual representation of the feature map and is frequently employed when the objective is to generalise features to a greater extent than preserving sharp edges [4].

Global Pooling methods, such as global max pooling and global average pooling, condense the entire feature map into a single value for each channel, based on the method chosen, e.g. if the global max pooling is chosen, entire channel is condensed into the maximal value of the channel. This technique is commonly employed at

the conclusion of a convolutional neural network, particularly in image classification tasks, where the spatial location of features is of lesser consequence than their mere presence. Global pooling mitigates the risk of overfitting by producing a compact representation of the input data, as evidenced by Ruby (2020). [4].

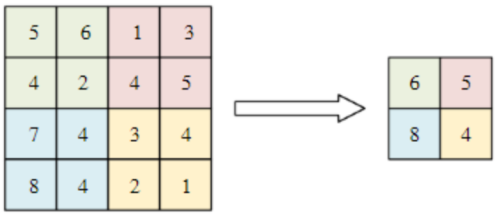


Figure 3: Pooling layer operation [5]

2.1.2 Activation Functions

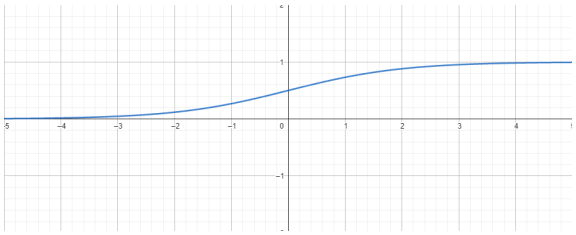
Activation functions are employed in convolutional neural networks and other neural networks to introduce non-linearity into the model, thereby enabling it to learn complex patterns and relationships within the data by determining the output of each neuron based on its input. After calculating the weighted sum of the inputs to a neuron, the activation function transforms this sum into the output. These functions allows the model to represent a wide range of functions. The output of the activation function serves as the input to the next layer.

In light of the work conducted by Siddharth Sharma and Simone Sharma regarding activation functions [6] the most prevalent activation functions are ReLU, Sigmoid, and Tanh (Hyperbolic tangent). Although alternative activation functions, such as BSF and ELU, could be employed, they are not utilised by the examined model (Yolov8). Consequently, a detailed discussion of these functions will not be provided. The following paragraphs will discuss the ReLU, Sigmoid, and Tanh activation functions based on the work of Siddharth and Simone Sharma [6].

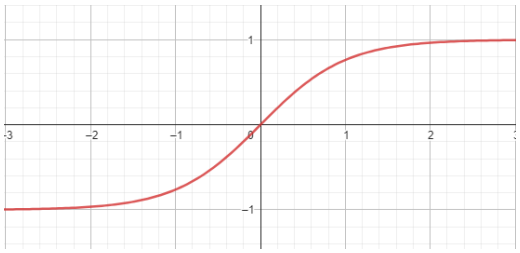
ReLU: the Rectified Linear Unit (ReLU) is one of the most common and easy to understand activation functions in convolutional (or any other) neural networks. It is defined as $f(x) = \max(0, x)$ in the works of Siddharth Sharma [6].

This function introduces non-linearity while maintaining a simple and efficient computation. The ReLU function helps mitigate the vanishing gradient issue, allowing models to learn faster and perform better.

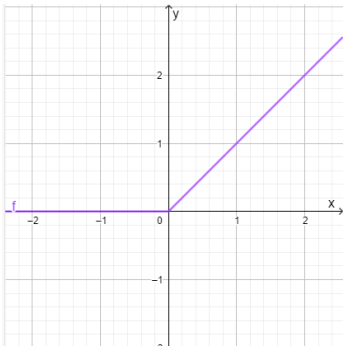
However, be susceptible to the phenomenon known as the ”dying ReLU” problem, where neurons become inactive and only output zeros, particularly during the training of deep networks.



(a) Sigmoid activation function



(b) Tanh activation function



(c) ReLU activation function

Figure 4: Activation functions: Sigmoid, Tanh and ReLU

Sigmoid: the Sigmoid function translates input values to a range between 0 and 1, rendering it useful for binary classification problems. It is defined as $f(x) = \frac{1}{1+e^{-x}}$ in the works of Siddharth Sharma [6].

While the Sigmoid function provides smooth gradients, it is prone to the vanishing gradient problem, especially for large positive or negative input values.

This can slow down the training of deep networks, which is why it is often replaced by other activation functions in hidden layers, though it still finds use in the output layer for binary classification tasks.

Tanh: the hyperbolic tangent (Tanh) function is another activation function that maps input values to a range between -1 and 1. It is defined as $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ in the works of Siddharth Sharma [6].

The Tanh function is zero-centered and centrically symmetrical, facilitates the centring of the data and may result in accelerated convergence during training.

Nevertheless, it continues to exhibit the vanishing gradient problem for large input values, though to a lesser extent than the Sigmoid function.

2.2 Introduction to YOLO

The YOLO (You Only Look Once) model is a relatively recent object detection system that is straightforward to utilise and comprehend, while also benefiting from a thriving community of users and developers. It is based on the YOLO algorithm, which is a real-time object detection algorithm developed by Joseph Redmon and Ali Farhadi in 2015[7].

Unlike traditional object detection methods that apply classifiers to various sections of an image, YOLO (You Only Look Once) approaches the problem as a single regression problem. The algorithm divides the input image into an $S \times S$ grid and predicts bounding boxes and class probabilities in parallel for each grid cell, enabling the model to detect multiple types and instances of objects in a single run. This kind of architecture not only enhances speed by processing the entire image at once but also improves detection accuracy by reducing the number of false positives. By minimizing redundant detections and effectively capturing the spatial context of objects, YOLO allows for real-time applications, given its fast inference speeds, making it particularly effective in scenarios such as video surveillance, autonomous driving, and robotics.

The YOLO model has evolved through multiple versions, with improvements in both performance and varying capability. Subsequent versions of the model, including Yolov2, Yolov3, and the latest Yolov5 and Yolov7, as well as Yolov8 (with Yolov10 and 11 forthcoming), have introduced advancements in network architecture, feature extraction, and training techniques. These developments have rendered YOLO a suitable candidate for a number of applications, including autonomous vehicles, as evidenced by my own observations during my work in the industry, surveillance systems, and real-time video analysis.

A further noteworthy attribute of the Yolo-type neural networks is their scalability and versatility in terms of model architecture. These models are available in a range of sizes (*nano, small, medium, large, extralarge*) and with a variety of detection types (*semantic segmentation, bounding boxes, oriented bounding boxes, instance segmentation*), which can be deployed in diverse scenarios.

Model	size (pixels)	mAPval 50-95	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	params (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.2	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

Figure 5: Different sizes of the Yolov8 [8]

2.2.1 The role of model size

In the field of computer vision, particularly in the context of object detection tasks, the selection of an appropriate size for a Yolov8 model, such as Yolov8-Nano, Yolov8-Small,Yolov8-Medium, Yolov8-Large, or Yolov8-eXtra-large is of paramount importance in order to achieve an optimal balance between accuracy, speed, and resource efficiency.

The smaller models are lightweight and operate at high speeds, rendering them optimal for real-time applications on devices with constrained computational capabilities, such as mobile phones or edge devices. Although they consume less memory and processing power, they may compromise some accuracy, which can be an acceptable trade-off in applications where speed is more important than precision.

In contrast, larger models demonstrate superior accuracy and are capable of handling intricate image details with greater precision, through the models large number of trainable and frozen(not trainable) parameters. They are better suited to scenarios where there are fewer computational constraints, such as high-powered GPUs or cloud environments, where precise detection is essential.

Selecting an appropriate model size allows developers to tailor solutions to the specific needs of their applications, ensuring both cost-effectiveness and reliability. This careful selection directly influences the efficiency and effectiveness of computer vision systems, optimising them for diverse deployment environments.

In light of these considerations, I have opted to utilise a medium-sized model, aiming to strike a balance between computational capacity and accuracy.

2.3 Model architecture

This network, like many others in the CNN family, has a distinctive architectural configuration that enables the execution of intricate tasks such as object detection and classification. The system is constituted of three distinct and discrete components, each comprising a unique set of layers that perform specific and separate functions. The following paragraphs aim to provide an overview of these elements and their contribution to the model's detection.

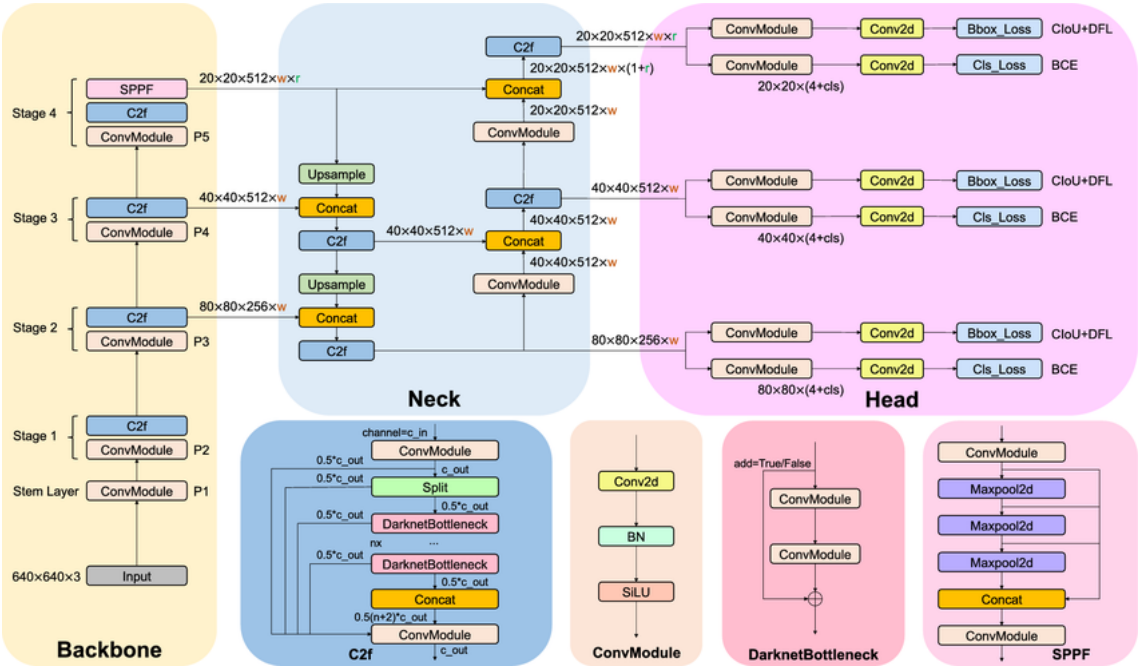


Figure 6: The architecture of the Yolov8 model [9]

Backbone: the Yolov8 model is based on convolutional neural networks that have been specifically designed to capture essential features from input images. It compri-ses multiple layers of convolutional operations (called Conv and a complex layer called C2F) that extract progressively more sophisticated representations. This structure emphasises both depth and computational efficiency, enabling the model to discern subtle details while maintaining rapid processing capabilities. Innovations such as skip connections and normalization techniques are employed to enhance the learning dynamics and improve the model's robustness to variations in input conditions.

Neck: in the Yolov8 architectural design, the neck serves as an intermediary between the backbone and the head. The primary function of the neck is to consolidate features from the various levels of the backbone, thereby enhancing the model's ability to detect objects across different scales. By employing strategies such as feature

fusion or pyramid pooling, the neck effectively integrates both coarse and fine features, enabling the model to better handle overlapping objects and diverse scene contexts. This feature aggregation is crucial for optimising the detection performance, as it allows the model to harness a comprehensive range of information.

Head: the head of the Yolov8 model is responsible for generating the final outputs, which are based on the features that have been processed through the neck. The final stage of the Yolov8 model translates the aggregated feature maps into bounding box predictions and associated class scores for each detected object. This section typically employs a combination of convolutional and fully connected layers to refine these outputs, ensuring they are accurate and meaningful. Additionally, the head may implement techniques such as adaptive anchors or confidence scoring to enhance the localisation and classification accuracy. By effectively synthesising the rich feature information, the head enables Yolov8 to achieve high-performance object detection suitable for a wide array of applications.

2.4 Training

The training process [7] for the Yolov8 model involves feeding its input with a large dataset of labelled images. The model learns to identify objects by minimizing the difference between its predictions and the actual labels trough multiple iterations: After a training cycle, if the default settings are kept, a validation function (so called "val") is run, to determine more information on the model's performance on pictures that are not present in the training set. The training process is iterative, with the model adjusting its parameters to improve its predictions over time. This process is computationally intensive and requires access to powerful hardware, the best fitting hardware for this purpose, which can be found in an ordinary PC, is the GPU.

The training process involves several key steps, including data preprocessing, model initialization, loss calculation, and parameter optimization. The model is trained using a technique called backpropagation, which involves adjusting the model's weights based on the error between its predictions and the ground truth labels.

The Yolov8 model uses a number of different loss functions, according to the original paper [7] , loss functions to measure the difference between its predictions and the actual labels in different ways:

- **CIoU (Complete Intersection over Union) loss**:

$$\text{CIoU} = 1 - \left(\text{IoU} - \frac{\rho^2(\mathbf{b}, \mathbf{b}^g)}{c^2} - \alpha v \right) \text{ [10]} \tag{1}$$

  where ρ is the Euclidean distance between the centre-points of the predicted box $\mathbf{b}$ and the ground truth box $\mathbf{b}^g$, c is the diagonal length of the smallest enclosing box covering the two boxes, α is a positive trade-off parameter that serves to balance the importance of the aspect ratio consistency term v, that measures the consistency of aspect ratios. It is more sensible to localisation accuracy.

- **DFL (Distribution Focal Loss)**:

$$\text{DFL} = -\sum_{i=1}^{N} p_i \log(q_i) \text{ [11]} \tag{2}$$

  where p_i is the true probability distribution and q_i is the predicted probability distribution. It is sensible to the accuracy classification.

- **BCE (binary cross-entropy for classification loss)**:

$$\text{BCE} = -\sum_{i=1}^{N} y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \text{ [12]} \tag{3}$$

  where y_i is the true label and p_i is the predicted probability.

I opted for training the model on my local machine, using a single GPU, while it provided me with sufficient performance, the training time was significantly longer than it would have been on a more powerful cloud machine. To reduce the chance of overfitting, the training dataset is typically divided into training and validation sets, with the latter used to evaluate the model's performance on unseen data. The trainings batch size where set to 3, which is not common, but it was necessary to fit the model on the GPU, to optimise training time.

3 Object detection component implementation

3.1 Dataset and formats

My project[1] started with choosing and downloading a suitable dataset containing data from urban traffic scenarios. I chose the Cityscapes dataset [13].

These data was not in the correct format for the Yolo model to use. The dataset contains semantic segmentations, but the vanilla Yolo architecture is working with bounding boxes. I needed to write a format conversion script, following the descriptions in subsection 3.2.

3.1.1 Cityscapes overview

The Cityscapes dataset is a large-scale dataset [13] used for training and evaluating object detection models. It contains high-resolution images of urban scenes, with detailed annotations for various objects such as cars, pedestrians, and traffic signs. The dataset is widely used in the field of computer vision for tasks such as semantic segmentation and object detection.

I chose the gtFine dataset, consists of precisely labelled segmentation masks. Based on their work, where they published this dataset [13] titled "*The cityscapes dataset for semantic urban scene understanding*" the gtFine dataset consists of 5000 images, with a resolution of 1024×2048 pixels, and annotations for 30 classes of objects. It is divided into three subsets: training set, validation set, and test set, with 2975, 500, and 1525 images, respectively.

The validation and training sets contain annotations for 30 classes, while the test set does not include any annotations. The dataset is labelled using pixel-level segmentation masks, which provide detailed information about the location and shape of objects in the scene.

However, for the purpose of this work, the annotations were converted into a bounding-box format, which is more suitable and straightforward for this object detection tasks. Also, the dataset was filtered to include only the classes that are relevant to the project.

[1] Impementation can be found on https://tinyurl.com/mrx6ante

3.2 Data engineering

The images and annotations are converted to a format that the Yolov8 model can process, which is a text file that contains the path of the image and the coordinates of the bounding boxes in pixels for each object in the image. The format is straightforward: each line of the text file represents a single instance of an object. The first variable is the label_id, which denotes the object's class. This is followed by the two-point's four (x, y) type coordinates in pixel. This process uses a custom script that reads the annotations from the Cityscapes dataset and converts the labels into labels whose classes are filtered and transformed the relevant classes into the grouping I chose for this project.

The classes were grouped into five categories:

- **small vehicle** *(usually cars, which are for personal use)*,
- **large vehicle** *(buses, trucks and other large non personal vehicles)*,
- **two wheeler** *(bicycles and motorcycles)*,
- **On-rails** *(trains and trams though the smaller Fine dataset did not include any in the training set)*
- and **person** *(pedestrian, and rider)*.

The categories were selected on the basis that the model's confusion is more readily managed when objects that are more similar in appearance are grouped together, thereby facilitating the model's ability to distinguish between them.

The script also converts the semantic segmentation masks into bounding boxes, trough finding the most extreme points and of the mask, creating a bounding box around them. This converted output is then saved into a text file, which is in the format used as input for the Yolov8 model.

The final structure of the dataset with metadata (descriptors and lists of labels and pictures) is as follows:

- **Root (gtFine)**: The root directory of the Cityscapes dataset, which contains the images and annotations.
  - **image**: Folder containing the images in the dataset, broken down into training, validation, and test sets and further divided into subfolders based on the city where the images were captured.
  - **labels**: The class label for each object in the image, with the same subfolder structure as the images.

- **train.txt**: The training set, which contains the paths to the training images and their corresponding annotations.
- **val.txt**: The validation set, which contains the paths to the validation images and their corresponding annotations.
- **test.txt**: The test set, which contains the paths to the test images and their corresponding annotations.

- **Descriptors**: The folder containing a list of class labels and their corresponding indices, as well as path set descriptor txt-s, the train-, val- and test.txts. It is used by the model to determine the classes, their indices and the paths to the images.

3.3 Model Implementation

3.3.1 Justification behind the selection of YoloV8

For my work, I chose the **Yolov8m** configuration, which stands for **Yolo version 8 medium**. My choice was made on the bases of previous experience with this model architecture and the popularity of its applications, made it so there are an abundance of literature regarding this specific algorithm and architecture. It is also suitable for numerous, different applications, this aspect was important because of the high variance in sizes and shapes of the traffic objects needed to be detected by the network. Training with custom datasets has also been made relatively easy.

I also examined other networks such as Faster R-CNN and even an older Yolo version, Yolov5. I tested Yolov5 on the same dataset and produced inferior performance KPIs (Key Performance Indicators). I confirmed my findings by a paper that was discussing an application of these models [14], Yolov5 exhibits inferior performance relative to Yolov8 on identical input data and with an equivalent architectural configuration. Additionally, it is also faster. This comparison renders the Yolov5 model obsolete, and therefore it has been excluded from further consideration.

The Faster R-CNN while being a less sophisticated model it gets outperformed by Yolo on, a comparison study on these networks "Analysis of the performance of Faster R-CNN and Yolov8 in detecting fishing vessels and fishes in real time" [15] it has a table which contains all the necessary information to decide that it is inferior to Yolov8 in almost every aspect, thus it was not chosen over the Yolo architecture.

Given its superior performance compared with the two models examined as well as the extensive availability of related materials, and it's ability to predict bounding boxes and class probabilities for objects in an image, the Yolov8 algorithm has been selected for this undertaking.

3.3.2 Chosen model's description and parametrisation

Based on paragraph 3.3.1 the Yolov8 bounding box detection medium version of the model was picked for this project. In order to facilitate the requirements of the project, the model was utilised in a pre-trained state, with the weights being downloaded from the official Yolov8 repository [8], trained on the COCO dataset, as described in the paper where they introduced the network [7]. Subsequently, the model was fine-tuned on the Cityscapes dataset, which was converted into a format compatible with the model's processing capabilities.

Training was conducted using the Adam optimiser with a learning rate of 0.01 for lr1 (initial learning rate) and lrf (final learning rate). I ran the training on the model for 30 epochs with a batch size of 3, to accommodate the limited local GPU memory. The training process ran on a single PNY NVIDIA Quadro P2000 5GB VCQP2000-PB GPU, with a total training time of 28.056 hours.

3.3.3 Training with the help of MLOps solutions

In order to monitor the performance of the model and to facilitate the visualisation of the learning process, as well as to organise the experiments, an online machine learning monitoring solution, Comet.ml, has been selected.

This tool is capable of tracking the training process in real time and of visualising the properties of the model on a user-friendly dashboard, which can be accessed from any device with an internet connection. The Comet.ml platform also provides a range of features that can be used to compare different experiments, such as hyper-parameter tuning, model versioning, and collaboration tools, while also offering a comprehensive set of APIs for integration with other tools and platforms.

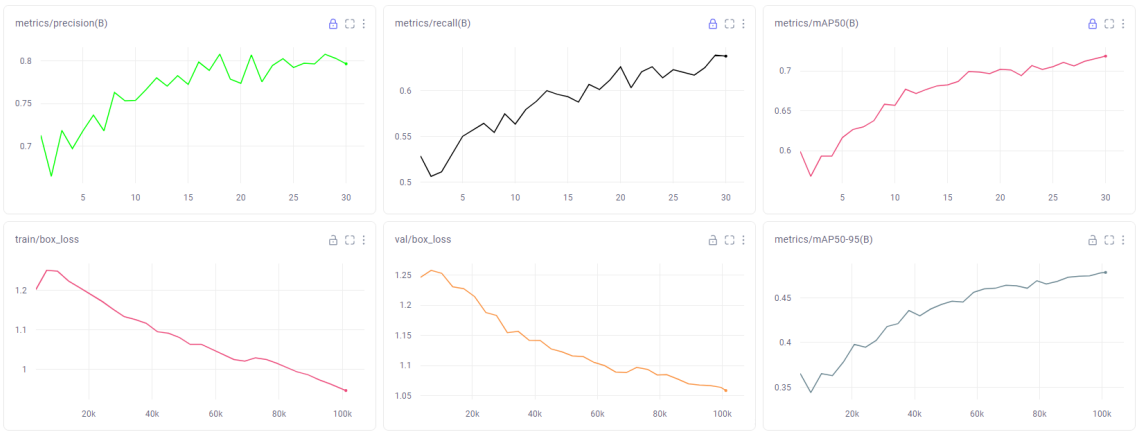


Figure 7: Comet.ml dashboard made from the training of our model.

Figure 7 depicts the Comet.ml dashboard, which offers a comprehensive representation of the training process, encompassing loss and accuracy metrics, the learning rate, and the model's performance on the validation set. This information is vital for evaluating the model's performance and for identifying potential issues that may arise during training.

Furthermore, the Comet.ml platform enables the comparison of different experiments, which can be beneficial for fine-tuning the model's hyperparameters and for improving its performance. Which I did try out, for comparing the performances of the same model on the same dataset, but with different epochs and batch sizes to see and find a good balance between training time and performance.

3.3.4 Model evaluation

The evaluation of the Yolov8 model is performed using a set of metrics that measure its performance on the Cityscapes dataset. These metrics include precision, recall, mAP, and IoU, which are commonly used in object detection tasks. I used the mAP metric to evaluate the model's performance on the test set, which provides a comprehensive measure of its accuracy. The mAP is calculated by averaging the precision-recall curves for each class in the dataset, which gives an overall measure of the model's performance.

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^{N} \text{AP}_i \tag{4}$$

where N is the number of classes and AP_i is the average precision for class i.

The IoU metric is used to evaluate the model's ability to accurately predict the bounding boxes for objects in the image. It measures the overlap between the predicted and ground truth bounding boxes, with a higher IoU indicating a more accurate prediction.

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}} \tag{5}$$

In order to ensure the reliability of the subsequent analysis, only those detections with an IoU value of at least 0.80 were considered, and only those predictions with a relatively high degree of accuracy were utilised.

The performance of the final and best performing model (which will be later used in the interpretation generating component) is evaluated on the test set's all classes averaged:

Class	Images	Instances	Box(P	R	mAP@0.5	mAP@0.5:0.95)
all	500	10654	0.797	0.638	0.719	0.478
small_vehicle	479	4832	0.889	0.751	0.834	0.604
person	441	4095	0.829	0.651	0.738	0.445
large_vehicle	141	191	0.729	0.597	0.666	0.511
two-wheeler	361	1536	0.741	0.553	0.637	0.354

Table 1: Network performance is evaluated across all classes, considering a detection valid if it achieves an IOU (Intersection Over Union) of at least 70%.

4 Model Interpretation

4.1 Importance of Interpretation

Interpreting machine learning models is essential for understanding their decision-making processes. It helps in identifying biases, improving model performance by providing information helping us the augment data more efficiently, and building trust with users. Interpretation techniques provide insights into how image processing models make predictions and highlight the most influential features.

Interpretation of machine learning algorithms is a fairly new field, but it has gained significant attention in recent years due to the increasing complexity of models and the need for transparency and accountability. This need comes form industrial and judicial actors, who require explanations for the decisions made by models, especially in safety-critical applications such as autonomous vehicles, healthcare, and finance. In the last couple of years, the field of model interpretation has seen significant advancements, each brought us closer to understanding the inner workings of machine learning models, and try to put reason and logic behind otherwise black-box models.

Furthermore, based on these advancements the aforementioned actors are more willing to adopt laws, standards and regulations that require models to be interpretable, if they are to be used in non-research areas.

Building on the aforementioned state of the machine learning world, I felt it was important to try and create a fairly coherent interpretation of the model to meet today's industry standards.

4.2 Introduction to Interpretation methods

Model interpretation is a critical aspect of machine learning that aims to explain the decision-making process of models. Interpretable models are essential for building trust with users, identifying biases, and with this information, developers have a deeper understanding that can help them improve model performance. In this section, there will be a discussion of the importance of model interpretation and a review of different methods of interpretation.

In their survey [2], Liang et al. highlight the significance of model interpretation techniques in providing insights from the model itself, and the output of the model, without knowing the inner state of the model.

Therefore, they sorted these techniques into two categories:

1. data driven or more commonly as model-agnostic methods
2. model driven or model-specific methods

The following sections will provide an overview of the two categories. In the event that one is used, an analysis of the algorithms will be presented in light of the implementation used in this project. This will include a comparison and contrast of their respective strengths and weaknesses.

4.3 Model-Agnostic Methods

Based on the work of Liang [2], model-agnostic interpretation methods can be divided into three categories:

1. Perturbation-based interpretation and a Game theoretic approach
2. Adversarial-based interpretation
3. Concept-based interpretation

As discussed in the work of Liang [2], these are designed to explain model predictions without relying on the internal structure of the model hence their name, making them applicable to a wide range of models. This insight may be more applicable to local examples, as the approach only provides explanations for the given data and not globally for the model itself. For image processing purposes, these methods are particularly easy to understand and implement.

Both the game-theoretic and perturbation-based interpretation methods involve altering the input data and observing the resulting changes in the predictions of the

model on the modified image, while Adversarial-based interpretation methods focus on generating small perturbations or adversarial examples that can significantly change the predictions, highlighting the model's vulnerabilities.

On the other hand, concept-based interpretation methods aim to link model behavior to high-level human-understandable concepts.

Regardless, I will discuss the different types of data-driven interpretation methods in the following paragraphs, each on the basis of Liang's work [2].

4.3.1 Perturbation-based Interpretation

Perturbation-based interpretation methods are model-agnostic techniques that explain model predictions by perturbing the input data. These methods generate perturbed samples by introducing noise (or masking portions of the image) to the input image and observing the alterations in the model's predictions. This masking is the main principle behind these methods.

The most common type of masking is occlusion, where parts of the image are covered to determine their importance for the model's output. It can be interpreted as a form of feature selection, where the model's output is evaluated based on the presence or absence of specific features.

In essence, perturbation-based methods can be conceptualised as an optimisation problem. Given an input vector x, a model function $f(x)$, and a predicted vector y, the objective is to identify which components of x contribute to the generation of the prediction. To accomplish this, the input features are systematically altered by introducing perturbations δx). This approach allows us to infer the importance of specific features by quantifying the extent to which the predicted vector $y = f(x+\delta x)$ is affected by the introduction of these perturbations.

Perturbation-based methods are focusing on "*local interpretability*", meaning they explain specific predictions rather than providing an overarching understanding of the model's behaviour. This local explanation is vastly inferior to global interpretability. Regardless, sometimes it can be extended, to provide broader insights into the models logic.

It works in a similar way to the human visual system and the way we perceive the visual world, so that by obscuring different parts of an object, we can significantly alter our own eye's perception of the object. By analysing the effect of perturbations on the model's output, these methods identify the most important features for a given prediction.

As a prominent implementation of this method is LIME (Local Interpretable Model-agnostic Explanations), which will be discussed in a following section 5.1, it was selected for use in the project due to its simplicity and effectiveness in interpreting the model's predictions.

There are multiple types of perturbations:

- **Noise insertion**: adding some type of noise (Gaussian or random) to parts of the input data, which can reveal the sensitivity of the model to small variations.
- **Blurring/Pixel Modification**: when working with image data, perturbations can be created by blurring parts of an image or modifying pixel intensities, which helps to understand how the clarity or sharpness of particular regions affects the prediction.
- **Feature Deletion for structured data**: remove or zeroing specific features, determining how significant that feature really is to the prediction.
- **Text Data Perturbations**: for text data, this involves removing words, phrases, characters, but it can also mean swapping tokens with synonyms to gauge their importance to the output.

Given the project's domain, I only used perturbations useful in image processing tasks, and these exclusively consists of modifying pixels and groups of pixels to determine their usefulness to the detection. In the following, I will discuss two variations of this standard perturbation-based approach.

Game theoretic approach with perturbation is a model-agnostic interpretation method that employs cooperative game theory to explain model predictions. This translates to decomposition of the input image into a number identical, equally-sized components, referred to as "features". These are processed further and a so called Shapley value is calculated to each feature, based on their contribution to the prediction. This will be projected onto the image, where each feature will be coloured according to this value, thus facilitating easier reading and comprehension. A detailed examination of this process can be found in reference 5.1, where the operational mechanics of this methodology will be elucidated.

Based on the work of Lundberg [16] and Liang [2], this method, namely SHAP (SHapley Additive exPlanations), is particularly useful for image processing tasks, as it provides detailed insights into the model's decision-making process. On grounds of the aforementioned, I chose to use SHAP in my project, as it offers a comprehensive and theoretically grounded approach to interpreting the model's predictions.

Adversarial-based interpretation methods, based on the work of Lian [2], represent a subtype of perturbation methods, the objective of which is to provide more robust interpretations. These methods address a common issue in deep neural networks (DNNs), namely poor generalisation performance, as highlighted in the base of my work [2]. Deep neural networks (DNNs) are highly sensitive to perturbations in input data, which can result in significant alterations to the predictions made, thus affecting the stability and reliability of the interpretations produced. The earliest adversarial-based interpretation methods, as exemplified by the approaches put forth by Fong et al. (2017) and other researchers, employed techniques to mitigate the impact of perturbations. This was achieved by utilising random masks and regularising the masks with total-variation norms to smoothen the images that had been perturbed.

Although these methods enhance robustness, they necessitate manual tuning of hyperparameters and yield interpretations of relatively low resolution, thereby constraining their capacity for fine-grained analysis. On these grounds, they are not suitable for applications that require high-resolution interpretations, and such I did not use them in my project.

4.3.2 Concept-based Interpretation

Concept-based interpretation methods, based on the work of Liang [2], employ predefined sets of human-understandable concepts to provide explanations for deep learning models. The generation of these concepts is typically conducted with the assistance of domain experts, thereby rendering this method particularly useful for interpreting the decision-making processes of models in a manner that aligns with human intuition. The process commences with the selection of a concept set (C), which encompasses images or data that correspond to specific attributes of the input. For instance, the concept set may include images of tiger stripes for an image of a tiger.

The definition of these concepts is facilitated by humans, and the concept set is subsequently incorporated into the deep neural network (DNN) in order to ascertain which concepts are most relevant to the model's predictions. Two prominent concept-based interpretation methods are Testing with Concept Activation Vectors (TCAV) and Network Dissection (ND). TCAV explains the behaviour of the model in question by employing human-defined concepts, namely Concept Activation Vectors (CAVs) , which quantify the importance of said concepts to the model's predictions through directional derivatives. This approach allows for the assessment of the sensitivity of the model's predictions to specific concepts, such as texture or colour, across a range of inputs.

In contrast, Network Dissection quantifies the relationship between internal neural network features and visual concepts, utilising metrics such as Intersection over Union (IoU) to ascertain the degree to which individual neurons correspond to concepts such as colour, texture, or objects. This approach facilitates a comprehensive understanding of the specific layers and neurons in a CNN that are responsible for detecting various concepts, thereby offering insights into the interpretability of each layer.

Based on the work of Liang [2], these methods are particularly useful for image processing tasks, as they provide human-understandable explanations for the model's predictions. However, they require the input of domain experts to define the concepts, which can be time-consuming and may introduce bias. Furthermore, the interpretability of these methods is limited by the quality of the concept set, which may not capture all the relevant features of the input data.

As a result, I found that these methods were not suitable for my project, as they require a high level of domain expertise and may not provide the level of interpretability required for fine-grained analysis.

4.3.3 Comparison of chosen Model-Agnostic Methods

In this subsection, I will compare two of the most popular model-agnostic interpretation methods, LIME and SHAP. Based on the work of Liang [2], these model-agnostic interpretation methods have their strengths and weaknesses. Through their example I will illustrate the differences between the two methods.

LIME and SHAP while using different approaches, there are both using some form of perturbation to explain the model's predictions. LIME approximates the model locally by fitting a simple interpretable model around the prediction of interest, providing local explanations by perturbing the input data and observing the changes in the model's predictions. It is generally faster and simpler, making it suitable for quick, local explanations.

On the other hand, SHAP is based on cooperative game theory and uses Shapley values to attribute the contribution of each feature to the prediction. It provides both local and global explanations by considering all possible combinations of features, producing consistent and theoretically sound explanations.

Based on the discussions by [16], Kernel SHAP, a variant of SHAP, is particularly useful for image processing tasks, as it can handle high-dimensional data efficiently. It is based on the idea of approximating the model with LIME and using the Shapley values to explain the predictions the model made. This approach combines the

strengths of both LIME and SHAP, providing accurate and efficient explanations for image processing models.

However, SHAP is more computationally intensive due to the necessity of considering all potential feature combinations. During the execution of the interpretation scripts, SHAP exhibited approximately four times the GPU memory usage of LIME, while the runtime was approximately one hundred times longer. While LIME is flexible and can be applied to any model and data type, SHAP offers a more comprehensive and theoretically grounded approach at a higher computational cost.

4.4 Model-Specific Methods

The base idea behind this category is to ground our interpretation on the internal state of the model. By this in the field of image processing, it aims to somehow visualize some parameters of the inner state of the model, and project it back to the input image. Multiple methods exist in this category, such as Class Activation Maps, Gradient-based methods. The following paragraph, present a discussion of the Class Activation Maps with reference to the work of Bany Muhammad and his introduction of his interpretation method [17]. Its implementation will be discussed in the next section. Furthermore, I will discuss the Gradient-based methods based on the work of Selvaraju titled "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization" [18].

4.4.1 Class Activation Maps

Class Activation Maps (CAMs) are a model-specific interpretation method designed to highlight the regions or features in an image that are most pertinent to a given model. In an image, these are the regions that are most relevant to a model's decision, particularly in the context of classification tasks. The operation of CAMs is based on the projection of the activations from the convolutional layers back onto the input image. This results in the generation of a heatmap, visually represents the areas that contributed the most to the prediction [17]. This method provides valuable insights into which features the model is focusing on, enabling a better understanding of the model's decision-making process.

The original CAM technique, first introduced by Zhou et al. (2016), is specifically designed for models that include global average pooling layers before the final output layer, often referred to as the "head". These pooling layers facilitate and are responsible for the smooth projections (feature maps) on the methods output image. While the CAM technique is highly effective, it is somewhat limited in that it is optimized for this specific architectural configuration, which restricts its versatil-

ity. It is less flexible for networks that rely on fully connected layers following the convolutional layers [19].

To illustrate, in a classification task where a model identifies a vehicle in an image, CAM would generate a heat map over and around the vehicle. This indicates that the model primarily focused on that object when making its decision, which is a useful insight. In the absence of this information, there is a possibility that the network may be detecting a feature that is not exclusive to that particular object class and this can result in greater confusion within the model.

This type of visual feedback is especially valuable in domains such safety critical (automotive, etc.) or medical fields, where understanding why the model identifies certain patterns is critical [17].

In response to the architectural limitations of the original CAM method, several improvements have been proposed. One significant extension is Grad-CAM (Gradient-weighted Class Activation Mapping), which eliminates the dependence on specific model architectures by using gradients to compute the heatmaps. Details of Grad-CAM will be further discussed in the next section [18].

4.4.2 Gradient-based Methods

Gradient-based methods, such as Grad-CAM (Gradient-weighted Class Activation Mapping), take a different approach by utilizing the gradients flowing through the backpropagation of the network. It can be used to localize the features within the image, that the models learn on. Grad-CAM generates heatmaps that highlight the regions of the image that contributed the most to the model's output. Selvaraju et al. (2019) demonstrated the effectiveness of this technique in their work titled "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization" [18].

By leveraging gradients, Grad-CAM provides more precise localization of features compared to basic CAM techniques, making it particularly useful for deep neural networks in image recognition tasks.

At the end, this method, despite of being more accurate than the vanilla EigenCAM, I did not use it in my project due to two main considerations.

First of all, the method is more computationally expensive, and requires more memory to run.

Secondly, the method is more complex to implement and requires more input from the model's internal state. In order to ensure the model calculates the gradients

correctly, it should be fed with the whole back-propagation chain, which has made the other interpretation methods more time-consuming to implement and run.

Based on that I chose to use the CAM method in my project, as it is simpler to implement and provides valuable insights into the model's decision-making process.

4.4.3 Comparison of Model-Specific Methods

Both CAM and Grad-CAM methods have been widely adopted for interpreting convolutional neural networks and are valuable tools for visualizing and understanding model behaviour. While CAM is limited by its dependence on specific model architectures, Grad-CAM offers a more flexible and generalizable approach that can be applied to a wide range of models. The use of gradients in Grad-CAM allows for more precise localization of features within the image, providing valuable insights into the decision-making process of the model.

In contest of the resulting visualizations, the output of the two networks tends to be really similar to each other. Which can be because the two methods are based on the same principle, and the implementation of the Grad-CAM is a direct extension of the CAM method, utilizing the activation maps with the gradients to provide the explanation.

4.5 Comparison of Interpretation Methods

In this section, I will compare the model-agnostic and model-specific interpretation methods discussed and chosen in the previous subsections and paragraph. Model-agnostic and model-specific interpretation methods each offer unique strengths and weaknesses when applied to an image processing model. Both approaches aim to provide insights into the model's decision-making. However, they are fundamentally different in terms of their underlying assumptions, flexibility, and computational requirements.

Model-agnostic methods often operate by perturbing the input data, observing the changes in the predictions of the model, and deriving explanations based on these observations as discussed previously in subsubsection 4.3.1. This makes them highly flexible and applicable to a wide variety of tasks and models. However, this flexibility sometimes comes at the cost of interpretability precision, as model-agnostic methods approximate how a model behaves based on perturbations, which may not capture the complete depth of the model's decision logic.

Conversely, model-specific methods are designed to align with the internal mechanisms of a specific model architecture, thereby facilitating the generation of expla-

nations based on the model's inherent operational logic. These methods are often more precise, as they can directly access the model's components — such as layers, weights, activation vectors, or gradients and utilize this information to interpret how the model made its decision. In fields like image processing, model-specific methods, such as Class Activation Maps (CAMs) or Gradient-based approaches, provide visual insights into the areas of an image that most influenced the model's predictions. By visualizing the internal state of the model, model-specific methods offer a more direct way of understanding the behaviour of the model, especially in tasks where identifying critical features or regions of the data is essential.

In comparing these two categories, the fundamental difference lies in their generalization versus precision. Model-agnostic methods are particularly useful because of their ability to work across different models and domains, making them versatile tools in scenarios where multiple model types and architectures are used. However, they frequently offer a higher-level view of the behaviour of the model, which may be less precise than the insights gained from model-specific methods. Conversely, model-specific methods provide a more detailed data for understanding of the decision-making process, directly interacting with the internal components. This allows for more detailed insights but limits the applicability to certain model types.

5 Model interpretation component

Implementing the second component, which was responsible for the explainability of the model, my tasks were to implement and use the interpretation methods discussed in the next chapter. In order to run them I had to choose data, to feed to the models input. I chose the test set's pictures taken in the German city of Bonn, because it was the smallest set of pictures in the test set, which set was not run on the model before, to optimise the runtime.

The implementation of the different methods were discussed in their own paragraphs in the next subsection.

After the successful implementations of these methods, two of them got implemented in the main XAI Python script. Because of hardware limitations I decided to use an external, cloud solution to run SHAP.

After that I ran the python script to produce results for LIME and EigenCAM. Then on the pre-processed pictures and model I ran the notebook containing SHAP, and its output was downloaded into my local machine.All the implementation of the project can be found in my Github repository(https://tinyurl.com/mrx6ante).

5.1 Methods for model interpretation

This component was implement into a python script file, which loaded the model from a ".pt" PyThorch[2] file and reads in the given image. After that process is complete, a quick resizing of the images is taking place, then the different interpretation methods are run after each other.

EigenCAM: this model-specific interpretation method employs the eigenvalues of the convolutional layers to generate class activation maps. The eigenCAM values are derived from the activation vectors of the various layers, which are employed in the calculation of the activation maps. Subsequently, the maps are projected back to the input image, thereby highlighting the regions that are of particular importance for the model's prediction.

The aforementioned regions are useful for understanding the output of each layer and the decision-making process of the model.

The output of the EigenCAM method can be interpreted as a heatmap, where the intensity of the colour represents the importance of the region for the model's prediction. By analysing these heatmaps for different layers of the same image, we can gain a deeper insight into the model's decision-making process.

I used the github repository of Jacob Gildenblat [20] , which contains an implementation of the method proposed by Zhou [19]. After that the needed functions got called in the main python script, on the preprocessed image and the loaded model, and got iterated over multiple layers.

Local Interpretable Model-agnostic Explanations: this model-agnostic, perturbation-based method employs a local surrogate model to approximate the behaviour of the original model, by approximating the model's decision boundary by perturbing an instance and fitting a simpler interpretable model (such as linear regression) to the modified samples The method is frequently employed for the purpose of elucidating individual predictions and elucidating the model's decision-making process. For instance, in image classification, LIME can identify which pixels or regions (segments or superpixels) contribute most to the model's decision, while in other uses, for example in texts, it can highlight important words or phrases.

In my implementation of XAI component, superpixels are generated for LIME, using the Slic algorithm, and a linear model is fitted to the perturbed data. The superpixel perturbation represents a more sophisticated approach than regular grid-based perturbations, as it is more likely to capture the relevant features of the image.

[2]https://pytorch.org

This is achieved by creating a non-regular shaped superpixel out of similar and/or approximate pixels. Subsequently, the linear model, which is a white-box model, is employed to approximate the behaviour of the original model and provide an explanation for the prediction.

The output of the LIME method is a set of coefficients that represent the importance of each feature for the prediction of the model. By employing these coefficients and superpixels, the superpixels that are most crucial for the prediction can be identified by the colour of the given superpixels on the methods output image. Useful superpixels are green, while harmful superpixels, which impede the detection of objects, are red.

To implement and integrate this method, I wrote a small script, which used the LIME python package[3] to help with the correct implementation of LIME, after that I called the function containing all the features regarding LIME explanation in the main interpretation component called Yolov8_XAI.py.

Shapley Additive explanations: this model-independent, perturbation-based method employs cooperative game theory to assign an "importance" value, known as the Shapley value, to each feature, representing its contribution to the model's prediction. This approach is more intricate but nevertheless satisfactory than the LIME method. The image is partitioned into equal-sized segments, which collectively form a grid, these segments are then organised into a set. This set and all its subsets gets perturbed individually and the model's prediction is observed on them, based on these predictions they get assigned a Shapley value. This value is getting calculated from iteration to iterations for each segment, until all the subsets are evaluated. This calculation is based on the Shapley value formula.

In the implementation, I used Kernel SHAP, which can be described as Linear LIME combined with Shapley values calculated by the Kernel SHAP algorithm.

Theorem 1 (Shapley kernel).

The equations below are from [16].

$$\begin{aligned} \Omega(g) &= 0\text{: baseline value for model g,} \\ \pi_{x'}(z') &= \frac{(M-1)}{(\binom{M}{|z'|})|z'|(M-|z'|)}\text{: weight assigned to each subset of z',} \\ L(f,g,\pi_{x'}) &= \sum_{z'\in Z}\left[f(h_x^{-1}(z')) - g(z')\right]^2 \pi_{x'}(z') \end{aligned} \tag{6}$$

[3]https://pypi.org/project/lime/

where $|z'|$ is the number of non-zero elements in z'.

The loss function is employed to quantify the discrepancy between the model's prediction and that of the surrogate model.

The loss is calculated by taking the squared difference between the predictions of (f) and (g) for each subset (z').

As this method is model-agnostic, it can be employed to interpret the predictions of any model, regardless of its complexity. This can be a significant limitation when interpreting complex models with a large number of features.

Thus, the method was only operational on an online platform, Google Colab[4], through a Jupiter notebook environment, as it provides the necessary computational resources and a fitting platform for the SHAP algorithm to run efficiently. The part of the implementation is based on the notebook of Akshay Gupta[5], his code was rewritten and repurposed, mainly the structure of the code remained, the most heavy changes were made in the image processing, the output generation and the parameterisation of the used kernel SHAP function.

5.2 Evaluation of interpretation methods

In all of the paragraph, the chosen methods will try to explain on the image on the Figure8.

Figure 8: Picture Bonn35

On which the models prediction can be observed in the image: Figure9.

[4]https://tinyurl.com/shapColab

[5]https://github.com/akshay-gupta123/Face-Mask-Detection/blob/main/Notebooks/Shap.ipynb

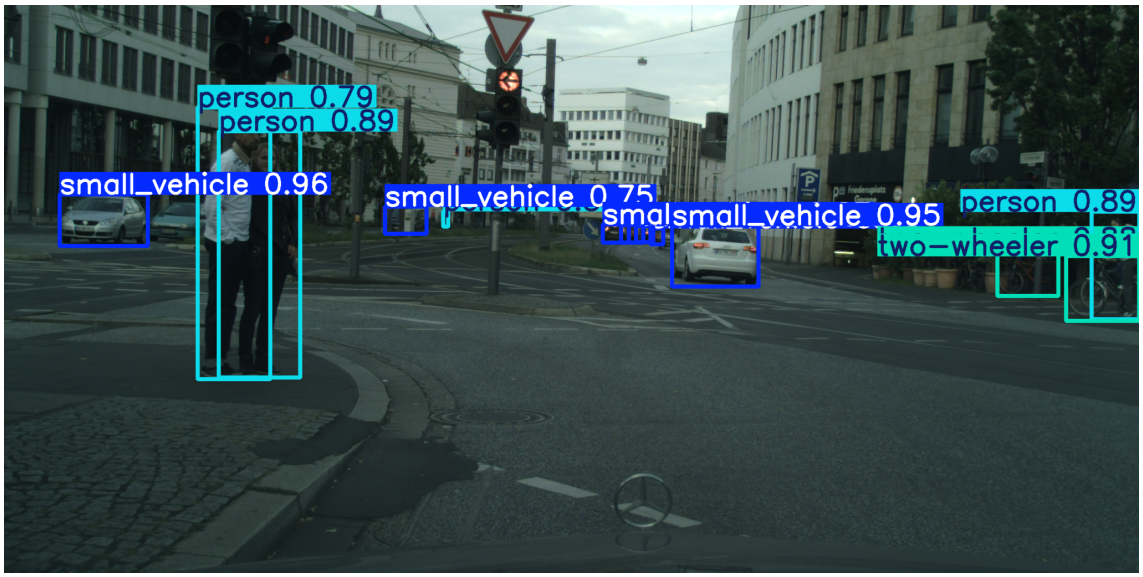


Figure 9: Detection of objects in picture Bonn35

In the following I will describe all the image interpretations given by the different interpretation methods. And try and interpret the model's workings based on those outputs.

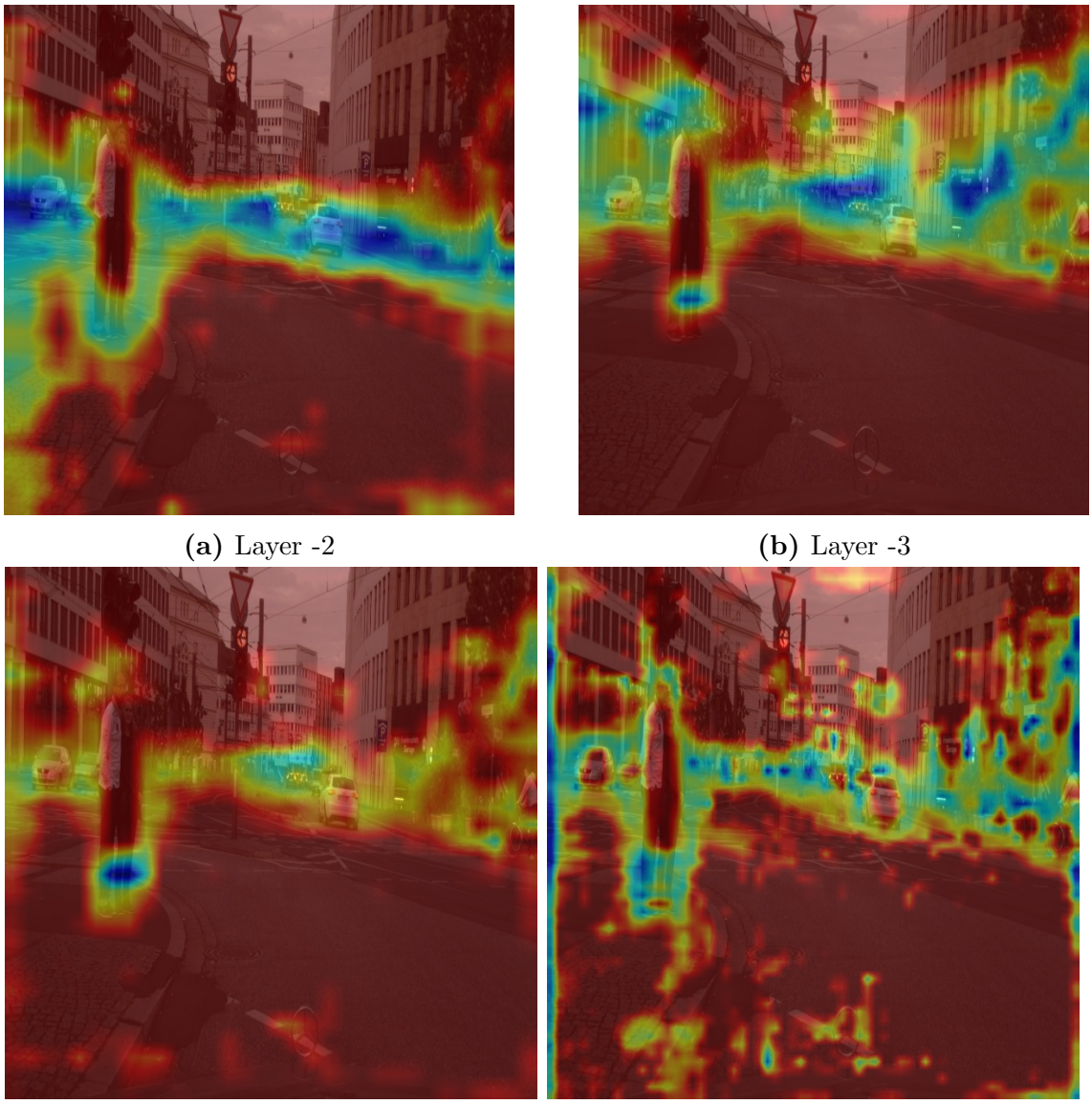
5.2.1 Evaluation of the EigenCAM method

In this figure constellation of Figure10 different layers are presented. These layers are

Name	Place	Description
-2 C2f	Feature Pyramid	A composite layer with Cross-Stage Partial Network (CSP) structure, designed to increase gradient flow and reduce memory usage by splitting the feature maps and merging them at the end.
-3 Concat	Feature Aggregation	Concatenates feature maps from different scales or layers to merge information, enabling multi-scale predictions.
-4 Conv	Convolutional Layer	A standard 2D convolution layer that extracts features from the input by applying filters to generate feature maps.
-5 C2f	Backbone Stage	Similar to the C2f layer at -2, it is responsible for feature extraction with a CSP-like structure to improve computational efficiency and gradient flow.

Table 2: Yolov8 architecture layers and their descriptions.

The images can be read and interpreted by identifying the layer responsible for each process and observing the colours on the provided activation maps. The colour red represents the lowest value of activation, while blue represents the highest. The



(a) Layer -2 **(b)** Layer -3

(c) Layer -4 **(d)** Layer -5

Figure 10: Activation maps for the Layer -2, -3, -4, -5 of Bonn35

range of layers observed spanned from the second to last layer to the fifth to last layer, allowing for the finest resolution of the head to be observed. This enabled the creation of a visualisation of the areas that were particularly useful in the detection process.

It is evident that the highest activation values were present in the regions where traffic-related objects were identified. Pedestrians were observed to have their outlines and legs presented with higher activation values. Additionally, it was noted that the rough edges from the more inner layers in the network exhibited a process of smoothening layer to layer.

5.2.2 Evaluation of the LIME method

In the figure constellation of Figure11 and 12 different types of LIME functions are presented. These are LIME_SUM and LIME_MULTI. LIME_MULTI aggregates the contributions of each feature across multiple instances by summing the importance scores. This approach provides a simplified, overall view of how certain features influence the decisions of the model on average, making it easier to identify the most significant features contributing to the predictions in a broad sense. LIME_SUM, on the other hand, focuses on explaining predictions at a finer level. It applies LIME to multiple instances individually and then analyses the distribution of feature importance across these instances. This method captures more granular information, allowing for a more detailed understanding of feature interactions and their varying contributions across different samples.

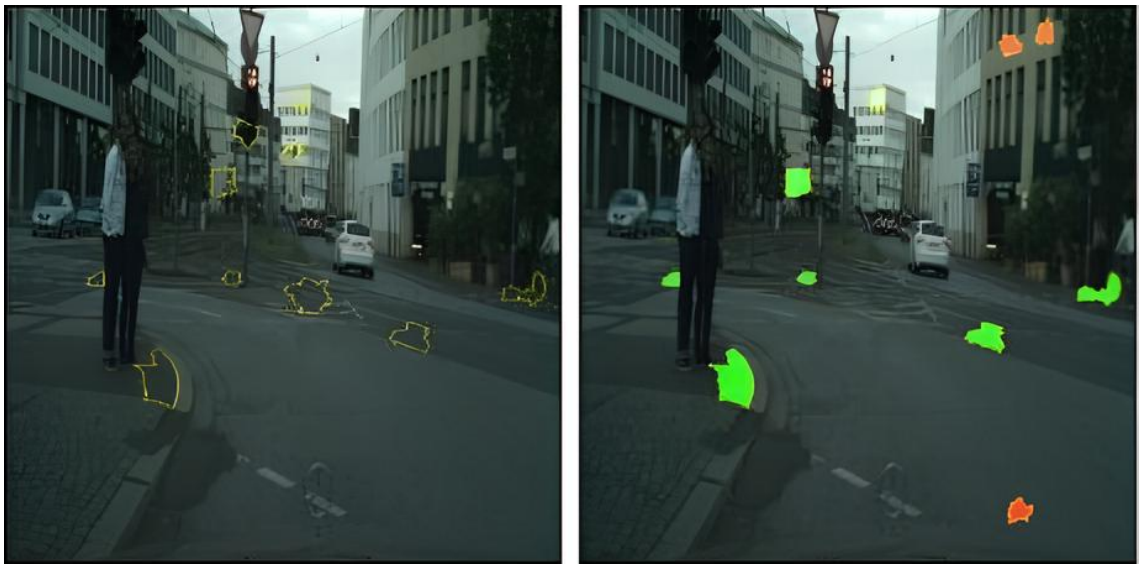


Figure 11: LIME_SUM

The method in question lacks the required granularity and is unable to keep up with more complex neural networks. Consequently, the resulting output is of questionable reliability, exhibiting inconsistencies such as the appearance of greens adjacent to detected objects, multiple artifacts, and a lack of discernible pattern or rationale in the red markings. This may be attributed to the method's inherent limitations in providing accurate explanations for intricate problems.

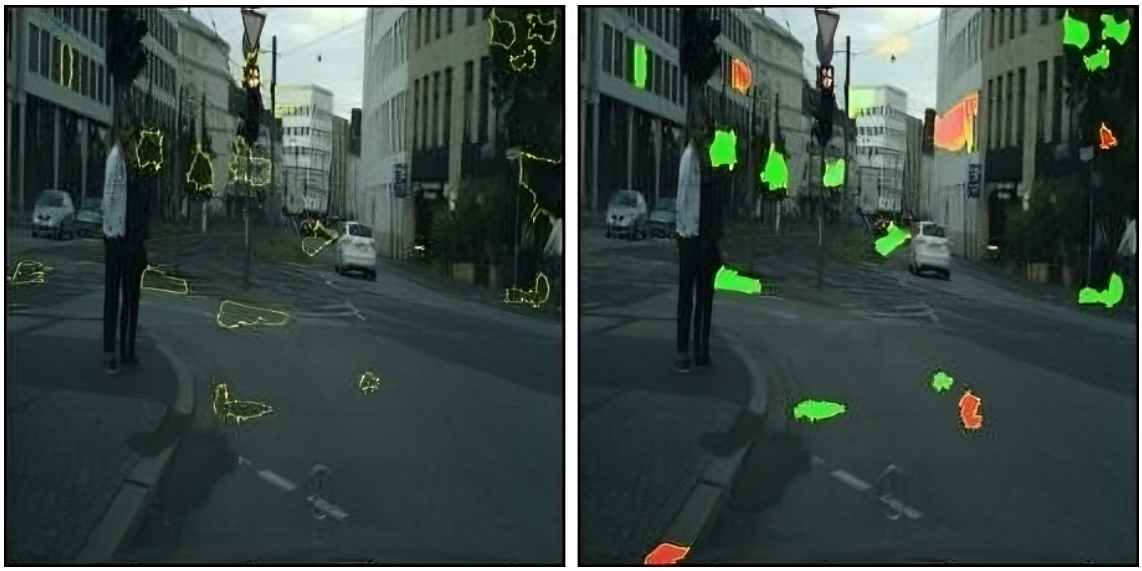


Figure 12: LIME_MULTI

5.2.3 Evaluation of the SHAP method

In the figure13 SHAP is run to the image, and the Shapley values are clearly visible on the pictures, each picture represents the run on one instance of detected object.

Red representing higher positive SHAP values, while blue represents negative SHAP values. Considering the most of the pictures, especially where overrepresented, highly visible objects can be seen, the interpretation is clean and readable.

The SHAP results in both rows (on Figure13 a and b) provide the following insights into the behaviour of the model in this traffic scenario:

- The uniform highlighting of pedestrians and vehicles across all images indicates that the model is effectively identifying and focusing on the critical elements within the traffic scene. This is an encouraging indication of the interpretability of the model and reliability in detecting important objects.

- The absence of emphasis on or attention to the muted or cooler blocks on buildings and other background areas indicates that the model does not rely on these features for its predictions. This is an anticipated and favourable outcome in the context of traffic-related tasks.

- Additionally, an anomaly is evident on the more ”noisy” images, which may be a limitation of this methodology.

(a) Shapley values of foreground objects

(b) Shapley values of background objects

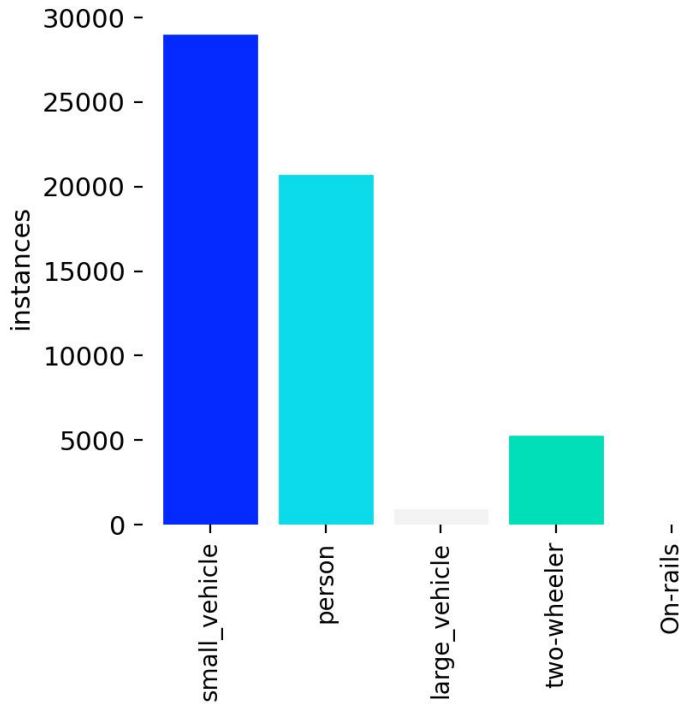
Figure 13: SHAP results for different instances in Bonn35

5.3 Addressing the Anomaly regarding the noise of SHAP and the probability of the models detection

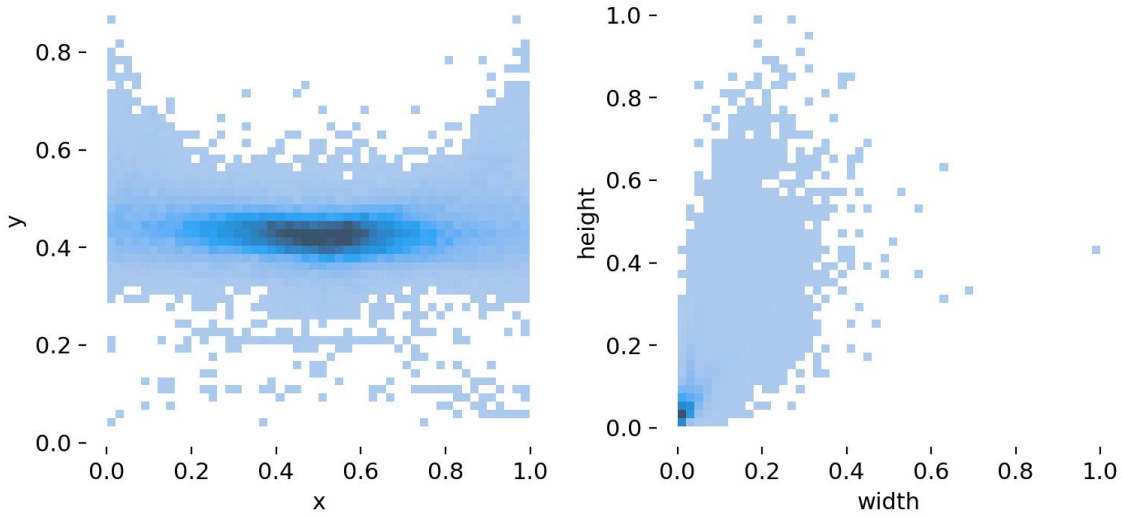
As previously stated in the previous subsection 5.2, the SHAP values for objects with lower detection probability are characterised by a high degree of noise, rendering them unable to provide a clear and coherent explanation for the predictions of the model. This anomaly can be attributed to the low probability of detection for this instance pedestrian class, which results in unreliable, often contradicting information for the SHAP algorithm to interpret. This phenomenon is common in object detection models, where the detection accuracy for small objects is lower than that for larger or more clearly visible objects. Furthermore, as previously discussed in section 4.3.1 that the detection threshold of this model and the probability of detection can have a significant impact on the actual decision-making process of the model. The combination of perturbation-based interpretation methods may result in unreliable and noisy results, as the perturbed image segments may be misclassified by the model.

Furthermore, an additional cause of this noise can be identified, which is more comprehensible when considering the proportions of the various classes of traffic objects present in the dataset. As illustrated in Figure 14 the dataset exhibits a significant class imbalance, with certain classes, such as vehicles, being overrepresented

in comparison to others, such as pedestrians. This imbalance may result in the generation of noise in the interpretations, as the model may exhibit a tendency to favour the more frequent classes due to the larger volume of data from which it has learned. Consequently, the limited number of examples for less frequent classes, such as pedestrians, may not provide the model with sufficient information to effectively capture their distinctive and often subtle features.



(a) Class distribution of labels



(b) Position and size distribution of labels

Figure 14

This deficiency in the training data can give rise to a number of issues. For example, the model may encounter difficulties in generalising effectively when it encounters pedestrian objects in real-world scenarios, due to insufficient exposure to their diverse appearances and contexts during training.

Consequently, the model may misclassify or fail to identify pedestrians, which can introduce additional noise in its output, manifesting as false negatives or incorrect predictions. (This is represented on Figure 15) Furthermore, the distinctive characteristics that differentiate pedestrian objects, such as specific shapes, movements, or interactions with the environment, may be under-represented in the model's learned representations.

To mitigate this noise and enhance detection accuracy, it is essential to balance the dataset or augment it with more diverse examples of the minority class, thereby enhancing the model's ability to learn these crucial features effectively.

Figure 15: Confusion matrix for the classification results of Yolov8 using the Cityscapes dataset.

The provided image presents two confusion matrices, one with absolute values and the other with percentages, which elucidate pivotal aspects of the model's performance.

To address this issue, an increase in the detection threshold of the model for the class of noisy object may be beneficial in improving the detection probability of already detected objects. This could be achieved by ignoring detections with a lower probability, thus enhancing the reliability and decreasing the noise of SHAP interpretations. This adjustment can help reduce the noise but may also result in the exclusion of some objects from the interpretation process. Making the interpretation process more incomplete, but more reliable and less noisy.

5.3.1 Explanations to the Confusion Matrix

The confusion matrices provide a detailed assessment of the model's classification performance for categories: *small_vehicle*, *person* and *large_vehicle*.

The absolute score matrix (left) shows high accuracy for *small_vehicle* (3,780 correct) and *person* (2,820 correct), but also reveals misclassifications, such as 640 *small_vehicle* instances identified as *background*. Similarly, the class *two-wheeler* has 928 correct classifications, but some confusion with *background*.

The percentage matrix (right) normalises these results and shows high accuracy for *small_vehicle* (85.06%) and *person* (77.49%), while *two wheeler* accuracy drops to 68.74% due to misclassification (30.29% as *background*). The *background* category also shows considerable confusion with other classes.

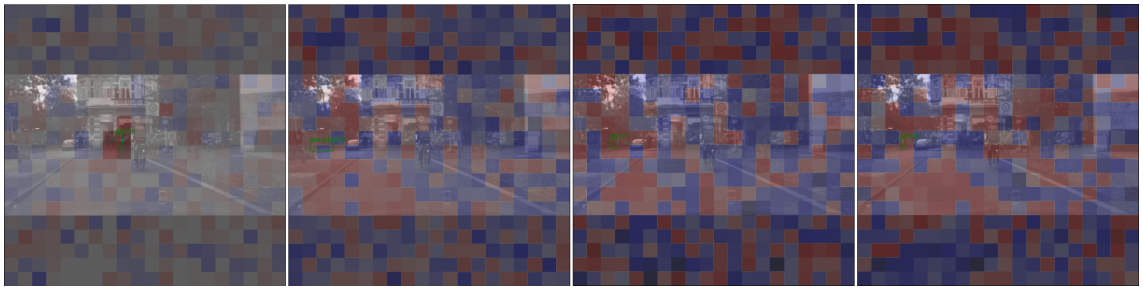
These matrices highlight strengths and weaknesses, guiding improvements in preprocessing, model design, and training while enhancing result interpretability—a core focus of this thesis.

6 Summary

(a) Shapley values of mostly visible objects in the centre of the picture

(b) Shapley values of partially visible objects

(c) Shapley values for different objects, the object on the side of the image has a low probability of detection, resulting in noise.



(d) Noisy Shapley values for partially occluded or too small objects

Figure 16: SHAP results for different instances

6.1 Further interpretation results

Explanation to SHAP (Figure16) Top Row (16a): The overlaid SHAP visualizations suggest which areas were crucial for the model in identifying these objects, which were two cars and the bike-rider. The highlighted regions are minimal, indicating a high-confidence detection of the object, with the model focusing specifically on the relevant parts of the image.

Top Middle Row (16b): Here, a broader range of the image is influenced by SHAP values, with more regions showing relevance to the model's predictions. This suggests that the model is incorporating additional contextual information, such as the surroundings or the road, to reinforce its detection decisions, as well as noise made by the lacking confidence detections of these objects. The highlighted regions expand around the objects, likely indicating the model's reliance on environmental cues in this analysis level.

Bottom middle Row (16c): In this row, the SHAP visualization becomes even more widespread, covering a significant portion of the image. This extensive coverage suggests that the model considers a broader context in its decision-making, potentially analysing both relevant and less relevant areas. This could mean that the model is being influenced by more peripheral or background features, reflecting a lower confidence level or a higher sensitivity to context.

Bottom Row(16d): These pictures are almost entirely made out of noisy explanations of SHAP, caused by the often discussed lack of confidence.

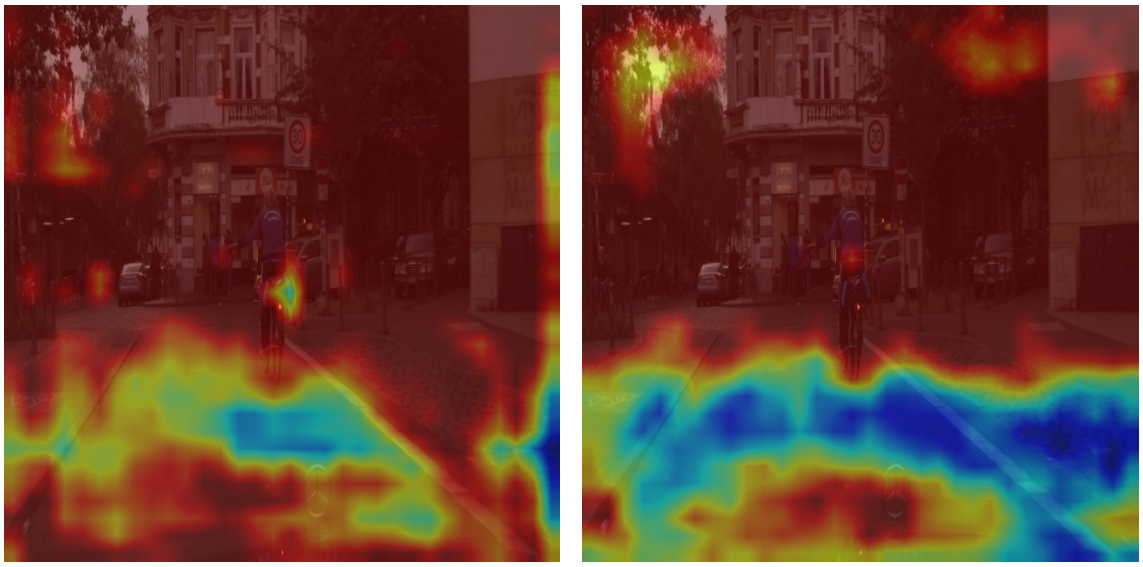
Explanations to EigenCAM results

1. *Layer -2 (17a)* : The activation map focuses on the central region, especially around the road and potential objects. The model detects general structures and outlines, emphasizing the primary objects in the scene.

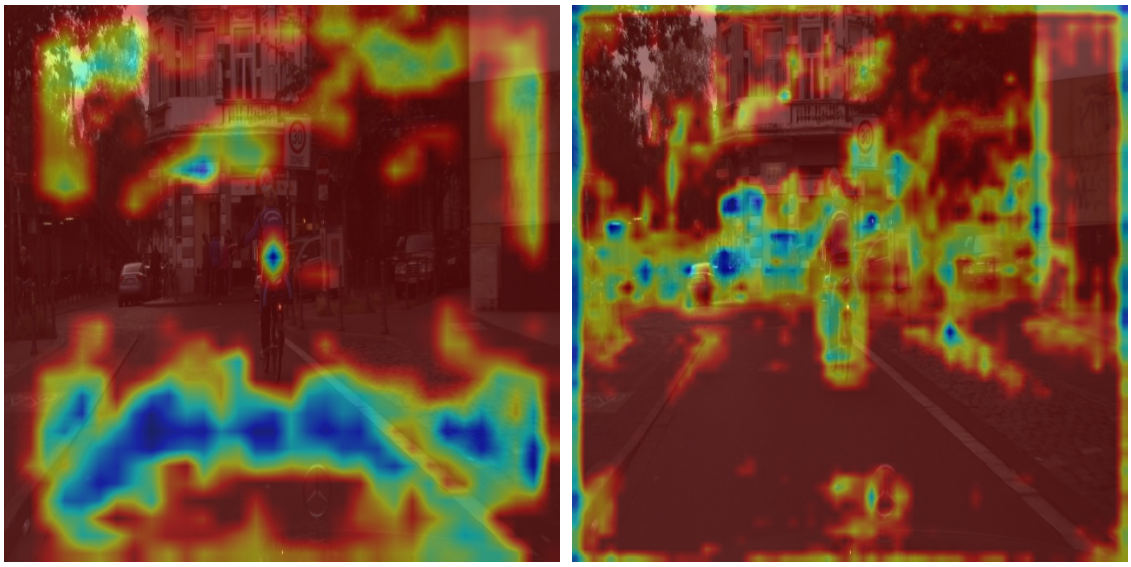
2. *Layer -3 (17b)* : The activation expands slightly, incorporating more background elements, such as buildings and signs. The model begins to consider the broader environment around the primary objects, adding context to the detection.

3. *Layer -4 (17c)* : The activations are more dispersed across the image, covering both foreground and background elements. The model captures finer-grained features, interpreting both the main objects and additional environmental details like signs and building textures.

4. *Layer -5 (17d)* : The activation map is broad and diffuse, with high responsiveness to nearly all elements on the image, including roads, buildings, trees,



(a) Layer -2 **(b)** Layer -3



(c) Layer -4 **(d)** Layer -5

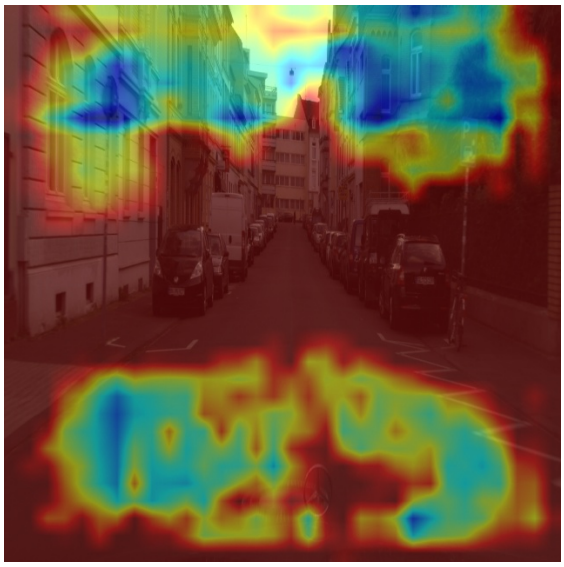
Figure 17: Activation maps for the Layer -2, -3, -4, -5 for Bonn36

and signs. The model at this stage captures complex, scene-wide patterns, and high-level feature representations.

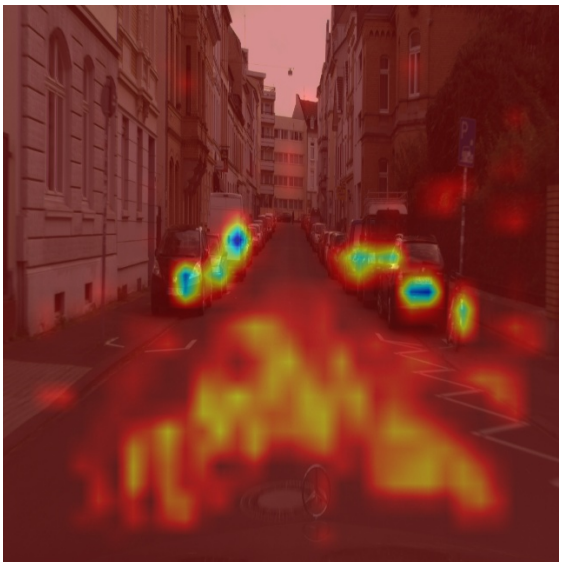
Explanations for Figure 18:

1. *Layer -2 (18a)*: The activation map shows focus on both the top and bottom regions of the image, capturing general structural elements, such as the sky at the top and the street at the bottom. This layer mainly highlights broad spatial features without much fine detail.

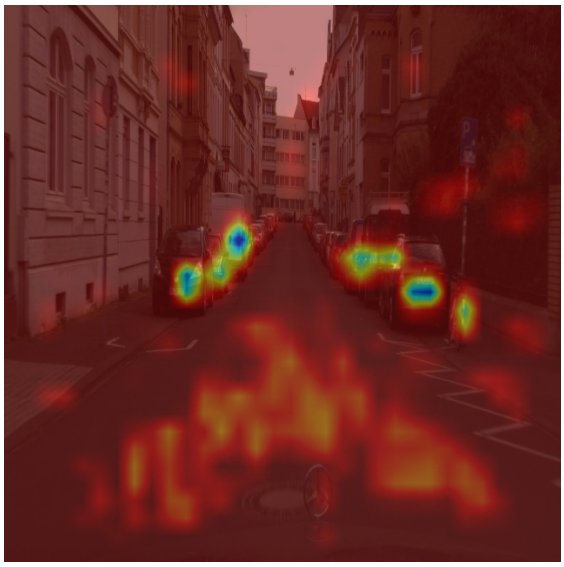
2. *Layer -3 (18b)*: Activations become more specific with a focus on distinct points along the street. This suggests that the model is beginning to identify



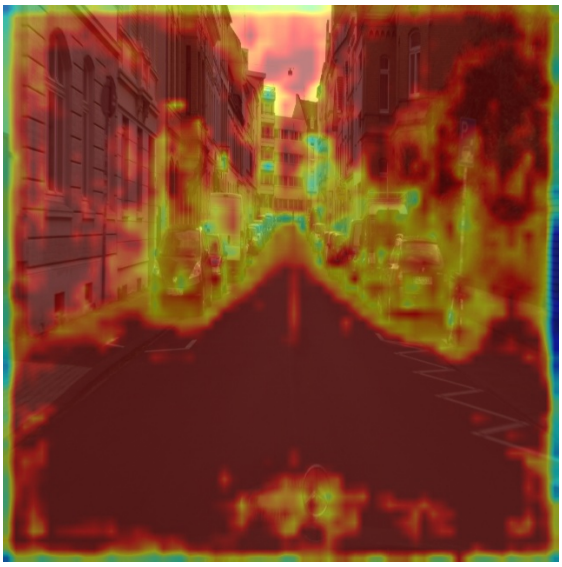
(a) Layer -2



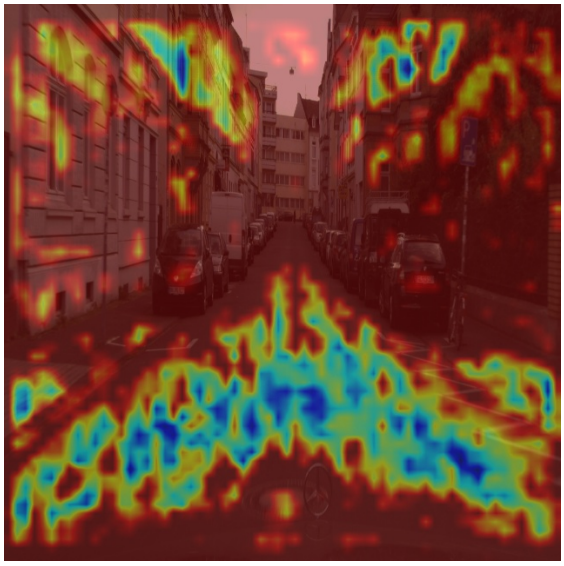
(b) Layer -3



(c) Layer -4



(d) Layer -5



(e) Layer -6

Figure 18: Activation maps for the Layer -2, -3, -4, -5 for Bonn37

specific objects or areas of interest along the roadway, possibly focusing on features relevant to the street layout or potential objects in the environment.

3. *Layer -4 (18c)*: The model's focus widens, capturing both sides of the street more prominently, with activations spreading across buildings and other elements lining the road. This indicates a balance between object-specific and context-specific features, as the model integrates more scene context.

4. *Layer -5 (18d)*: The activation is more expansive, covering almost the entire image, particularly emphasizing areas surrounding the street. This layer appears to interpret larger, scene-wide patterns, possibly identifying the overall structure of the street and surrounding buildings.

5. *Layer -6 (18e)*: Activations spread densely throughout the image, with a particular focus on the length of the street and the buildings that line it. This deep layer captures high-level abstractions, incorporating nearly the entire scene, reflecting the model's comprehensive understanding of the spatial layout and contextual details.

6.2 Takeaways

Notable advancements have been made in the interpretation and explanation of the behaviours exhibited by computer vision models, particularly through the application of SHAP and EigenCAM. These tools have facilitated a more profound understanding of the decision-making processes of intricate models, thereby offering a more transparent view of the manner in which these models prioritise and process visual information in different contexts. In particular, SHAP facilitated interpretability by decomposing model predictions into feature contributions. EigenCAM, on the other hand, used activation-based heat maps to visualise the model's spatial awareness.

While the results of LIME were less conclusive in this context, possibly due to challenges in its level of complexity compared to the model's architecture, the combined use of SHAP and EigenCAM successfully illustrated the potential for effective and detailed visualisations of model interpretations. By observing SHAP and EigenCAM outputs on the same image, we were able to achieve a more comprehensive view of model processing, providing a multi-faceted perspective on how the model detects and evaluates objects. This approach represents a step towards a comprehensive, global understanding of model behaviour through the integration of individual, local interpretations, where the strengths of each method complement the limitations of the other.

The robustness and accuracy of Yolov8 on the Cityscapes dataset further validated its ability to handle complex real-world scenes, particularly urban environments where object detection tasks can be challenging due to varying object scales, densities, and positions. Yolov8 demonstrated superior performance in accurately identifying a wide range of objects within the dataset - such as pedestrians and different vehicles - across different spatial arrangements, reflecting its resilience and adaptability to real-world applications. This consistency across different scenarios within the Cityscapes dataset underscores Yolov8's potential for use in applications that require high precision, such as autonomous driving and urban planning.

The successful application of Yolov8 in combination with interpretability methods such as SHAP and EigenCAM highlights a promising path for future computer vision models. This combined approach suggests that powerful models can also be interpretable, promoting transparency without compromising performance. This integration of interpretability tools with powerful models such as Yolov8 offers exciting possibilities for future work, such as real-time applications where both accuracy and transparency are paramount, ultimately contributing to the development of trustworthy, interpretable, and powerful computer vision systems.

6.3 Directions for further development

Although this project has made considerable progress in clarifying the functioning of models in the domain of computer vision, there are still numerous avenues for further investigation and development.

Firstly, enhancing the interpretability of underperforming methods, such as LIME, could improve the consistency and reliability of model explanations across various instances. Further investigation of parameter tuning or alternative implementations of LIME may facilitate the development of more robust explanations.

Furthermore, it would be beneficial to implement other, less well-known methods, or to revisit the approaches discussed in previous subsections (4), such as adversarial-based interpretation methods, for example Anchor [2]. Additionally, EigenGrad-CAM could be utilized or another model-specific method could be employed to gain a deeper comprehension of the model's internal operations by elucidating the training process.

Furthermore, the integration of interpretability techniques, such as SHAP and EigenCAM, into real-time applications represents a promising avenue for future research. Further work could concentrate on optimising these methods to operate effectively in conjunction with the model.

An expansion of the interpretability framework to include other datasets beyond Cityscapes would serve to further test the model's adaptability and the scalability of the interpretability methods across different environments. In addition, the incorporation of a more extensive range of data could facilitate a more comprehensive understanding of the generalizability and limitations of the model.

Finally, efforts could be directed towards the development of unified visualization tools that allow for dynamic, interactive exploration of combined interpretations from multiple methods. Such tools would enable researchers and practitioners to better analyse, understand and refine model behaviour across complex tasks, ultimately fostering greater trust and transparency in computer vision systems.

Acknowledgements

I would like to express my sincerest gratitude to my parents, Dr. Krisztián Nyilas and Ildikó Nyilasné Mészáros, for their unwavering support and encouragement throughout this academic endeavour. I would also like to express my gratitude to my girlfriend, Anett Bakos, for her understanding and assistance in maintaining my focus on my studies. I am extremely grateful to my consultant, Dr. Gábor Hullám, for his invaluable guidance and insights, which greatly enriched this work.

List of Figures

Bibliography

[1] ISO/IEC:5469:2021, "Artificial intelligence — functional safety and ai systems."

[2] Y. Liang, S. Li, C. Yan, M. Li, and C. Jiang, "Explaining the black-box model: A survey of local interpretation methods for deep neural networks," *Neurocomputing*, vol. 419, pp. 168–176, 2021.

[3] A. Das and P. Rad, "Opportunities and challenges in explainable artificial intelligence (xai): A survey," p. 1, 2020.

[4] J. Gu, Z. Wang, J. Kuen, L. Ma, A. Shahroudy, B. Shuai, T. Liu, X. Wang, G. Wang, J. Cai, and T. Chen, "Recent advances in convolutional neural networks," *Pattern Recognition*, vol. 77, pp. 355–359, 2018.

[5] S. Song, "The use of color elements in graphic design based on convolutional neural network model," *Applied Mathematics and Nonlinear Sciences*, vol. 9, 10 2023.

[6] S. Sharma, S. Sharma, and A. Athaiya, "Activation functions in neural networks," *Towards Data Sci*, vol. 6, no. 12, pp. 311–314, 2017.

[7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," 2016.

[8] UltraLytics, "GitHub - ultralytics/ultralytics at v8.2.103 — github.com." `https://github.com/ultralytics/ultralytics/tree/v8.2.103`. [Accessed 20-10-2024].

[9] R.-Y. Ju and W. Cai, "Fracture detection in pediatric wrist trauma x-ray images using yolov8 algorithm," *Scientific Reports*, vol. 13, 11 2023.

[10] X. Li, W. Wang, L. Wu, S. Chen, X. Hu, J. Li, J. Tang, and J. Yang, "Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection," pp. 4–5, 2020.

[11] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-iou loss: Faster and better learning for bounding box regression," *CoRR*, vol. abs/1911.08287, p. 4, 2019.

[12] U. Ruby and V. Yendapalli, "Binary cross entropy with deep learning technique for image classification," *Int. J. Adv. Trends Comput. Sci. Eng*, vol. 9, no. 10, p. 5396, 2020.

[13] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, "The cityscapes dataset for semantic urban scene understanding," in *Proc. of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[14] N. Chitraningrum, L. Banowati, D. Herdiana, B. Mulyati, I. Sakti, A. Fudholi, H. Saputra, S. Farishi, K. Muchtar, and A. Andria, "Comparison study of corn leaf disease detection based on deep learning yolo-v5 and yolo-v8," *Journal of Engineering and Technological Sciences*, vol. 56, p. 66, Feb. 2024.

[15] L. Ezzeddini, J. Ktari, T. Frikha, N. Alsharabi, A. Alayba, A. J. Alzahrani, A. Jadi, A. Alkholidi, and H. Hamam, "Analysis of the performance of faster r-cnn and yolov8 in detecting fishing vessels and fishes in real time," *PeerJ Computer Science*, vol. 10, p. e2033 9/15, 2024.

[16] S. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," pp. 4–6, 2017.

[17] M. B. Muhammad and M. Yeasin, "Eigen-cam: Class activation map using principal components," in *2020 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–3, IEEE, July 2020.

[18] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-cam: Visual explanations from deep networks via gradient-based localization," *International Journal of Computer Vision*, vol. 128, p. 336–359, Oct. 2019.

[19] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, "Learning deep features for discriminative localization," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1–3, 2016.

[20] J. Gildenblat and contributors, "Pytorch library for cam methods." `https://github.com/jacobgil/pytorch-grad-cam`, 2021.